

Protocolo de Trabajo con Git, GitHub y Linear

Taller de Integración II y Taller de Integración IV

Segundo semestre 2026

1. Objetivo

Esta guía fija la forma en que los equipos de Taller de Integración II (TI2) y Taller de Integración IV (TI4) trabajarán en el proyecto.

El flujo busca dos cosas concretas: que el trabajo se pueda seguir desde Linear hasta GitHub y que cada equipo conserve sus responsabilidades técnicas.

El proyecto usa tres herramientas:

- **Linear**: planificación, asignación y seguimiento del trabajo.
- **Git**: control de versiones.
- **GitHub**: alojamiento del código, Pull Requests, revisión de código e integración continua.

La división de responsabilidades es directa:

Linear gestiona el trabajo y GitHub gestiona el código.

2. Organización de los equipos

El proyecto estará compuesto por dos equipos.

2.1. Taller de Integración II

TI2 tendrá seis estudiantes y se encargará de:

- desarrollo de la aplicación web;
- desarrollo del backend y API mediante arquitectura por capas;
- implementación de los casos de uso y reglas de negocio compartidas;
- diseño e implementación de la base de datos;
- pruebas asociadas a sus componentes;
- despliegue de sus propios servicios;
- documentación técnica correspondiente.

2.2. Taller de Integración IV

TI4 tendrá tres estudiantes y se encargará de:

- desarrollo de la aplicación móvil;
- planificación general del proyecto;
- preparación y priorización inicial del trabajo;
- coordinación entre ambos equipos;
- control de calidad;
- revisión de código;

- seguimiento de dependencias;
- apoyo técnico e inducción al equipo TI2 cuando sea necesario.

TI4 debe ayudar a TI2 a resolver sus tareas cada vez con más autonomía. En una revisión, TI4 señala el problema y explica el criterio. La persona autora hace el cambio en su propia rama.

3. Repositorio GitHub

Todo el producto vivirá en **un único repositorio GitHub**. El monorepo contendrá las aplicaciones, el backend y los paquetes compartidos.

```
Proyecto
|
+-- .github/workflows/ Integración continua
+-- apps/
| +-- web/ Aplicación web de TI2
| +-- mobile/ Aplicación móvil de TI4
|
+-- convex/ Backend por capas y base de datos compartidos
| +-- presentation/ Funciones públicas de Convex
| +-- application/ Casos de uso
| +-- domain/ Reglas de negocio
| +-- infrastructure/ Persistencia y servicios externos
+-- packages/ Código compartido cuando corresponda
+-- package.json
+-- bun.lock
```

Cada equipo desarrolla, prueba y despliega los componentes que tiene asignados dentro del monorepo.

Un repositorio común no elimina la separación de responsabilidades. TI2 mantiene la aplicación web y el backend. TI4 mantiene la aplicación móvil.

La integración entre los equipos pasa por la API pública de la capa de Presentación del backend. Ambas aplicaciones consumen sus tipos generados. No se copian implementaciones ni se mantienen contratos externos.

3.1. Arquitectura por capas

Las dependencias deben avanzar en esta dirección:

```
Presentación
|
v
Aplicación
|
v
Dominio
^
|
Infraestructura
```

En el backend, las funciones públicas de Convex reciben la solicitud y llaman a un caso de uso. Los casos de uso coordinan la operación. El Dominio contiene las reglas. Infraestructura conecta con la persistencia, la autenticación y los servicios externos.

Una branch o Pull Request que cruce varias capas debe explicar qué cambia en cada una. React, Mobile y las funciones públicas no deben resolver necesidades de negocio con consultas directas a la base de datos ni con funciones extensas.

4. Uso de Linear

Todo el proyecto usará **un único workspace de Linear**.

Dentro de este workspace existirán dos Teams:

- **TI2:** Web, Backend y Base de Datos.
- **TI4:** Gestión y Aplicación Móvil.

Los dos Teams forman parte del mismo proyecto.

4.1. Triage

TI4 hará el *triage* inicial. Para cada issue debe:

- crear o revisar los issues necesarios;
- asignarlos al Team correspondiente;
- definir su prioridad;
- verificar que tengan una descripción suficiente;
- establecer criterios de aceptación;
- identificar dependencias entre ambos equipos;
- asignarlos al Cycle correspondiente.

Cuando un issue esté listo en el backlog, el equipo responsable organiza su ejecución.

4.2. Linear como fuente de verdad

Toda tarea técnica que requiera trabajo significativo debe tener un issue en Linear.

WhatsApp, Discord y otros canales sirven para conversar, no para asignar trabajo de manera formal. Si una conversación genera una tarea, alguien debe registrarla en Linear.

4.3. Sub-issues

El proyecto no usa **sub-issues** de Linear.

Para una tarea con varios pasos pequeños, usa una lista en la descripción del issue.

Por ejemplo:

TI2-37 Implementar autenticación

- Crear endpoint de login
- Validar credenciales
- Generar token
- Agregar pruebas
- Documentar endpoint

Si una parte del trabajo necesita asignación, seguimiento o revisión propios, conviértela en un issue independiente.

Así se evita gastar issues del plan en tareas que no necesitan seguimiento separado.

5. Cycles y Sprints

Cada *Cycle* de Linear representa un Sprint. No crearemos un Cycle por semana.

La correspondencia es esta:

Cycle 1 -> Sprint 1
Cycle 2 -> Sprint 2
Cycle 3 -> Sprint 3

Las semanas de cada Sprint quedan para seguimiento, reuniones y control de avance. Al cerrar un Cycle, el equipo revisa:

- trabajo completado;
- trabajo pendiente;
- issues bloqueados;
- deuda técnica generada;
- dependencias entre TI2 y TI4;
- trabajo que debe trasladarse al siguiente Sprint.

6. Estados de los Issues

El workflow tendrá pocos estados:

Backlog → Todo → In Progress → In Review → Done

También puedes usar:

Canceled

Los estados significan lo siguiente:

Backlog: trabajo identificado pero todavía no comprometido.

Todo: trabajo seleccionado para el Cycle actual.

In Progress: existe trabajo activo sobre el issue.

In Review: existe uno o más Pull Requests pendientes de revisión.

Done: todo el trabajo requerido por el issue fue integrado.

Canceled: el trabajo dejó de ser necesario.

Cuando sea posible, la integración Linear-GitHub cambiará estos estados de forma automática.

7. Integración de Linear con GitHub

El repositorio del monorepo debe conectarse al workspace de Linear.

Cada issue conserva el Team responsable, TI2 o TI4. El identificador distingue el área aunque todo el código viva en el mismo repositorio.

Los identificadores de Linear relacionan el código con el trabajo.

Por ejemplo:

TI2-37 TI4-18

Un Pull Request puede referenciar el issue desde la branch, el título o la descripción. La trazabilidad queda así:

Issue Linear → Branch → Commits → Pull Request → Review → Merge

8. Estrategia de ramas

El repositorio usará **feature branches pequeñas y de corta duración**.

La única rama permanente es:

```
main
```

No habrá una rama permanente **develop**. Tampoco usaremos Git Flow con ramas permanentes de desarrollo, release o hotfix.

8.1. Creación de una rama

Antes de empezar una tarea:

1. confirma que existe un issue en Linear;
2. actualiza la rama **main**;
3. crea una rama desde **main**;
4. trabaja solo en el objetivo del issue.

Flujo conceptual:

```
main
|
+-- rama TI2-37-login-web
|
+-- rama TI2-42-auth
|
+-- rama TI4-18-login-mobile
```

9. Nombres de ramas

Usa, de preferencia, el nombre de branch que genera Linear.

El nombre debe conservar:

- nombre o identificador del desarrollador;
- identificador del issue de Linear;
- una descripción breve.

Ejemplo:

```
vriviera/ti2-37-login
```

Si Linear genera un nombre demasiado largo, puedes abreviarlo.

Por ejemplo, en lugar de:

```
vriviera/ti2-37-implementar-endpoint-de-autenticacion-de-usuarios
```

usa:

```
vriviera/ti2-37-login
```

Conserva siempre el identificador de Linear.

No uses:

```
login
feature1
cambios
prueba
rama-nueva
```

10. Tamaño de las ramas

Una rama debe resolver **una intención concreta**.

Mantén las ramas pequeñas para facilitar:

- revisión;
- pruebas;
- resolución de conflictos;
- integración continua;
- detección de errores;
- reversión de cambios.

No fijaremos un límite artificial de líneas modificadas. El criterio es que otra persona pueda entender y revisar el Pull Request sin encontrarse con varias funcionalidades independientes.

11. Pull Requests

Todo cambio que llegue a `main` debe pasar por un **Pull Request** en el repositorio del monorepo.

Nadie puede hacer push directo a `main`.

11.1. Título del Pull Request

El título debe seguir la convención del repositorio. Revisa los Pull Requests recientes y el historial de Git antes de elegirlo. Con **Squash and Merge**, el título suele convertirse en el mensaje del commit que llega a `main`; por eso debe ser breve, claro y explicar el resultado del cambio.

Incluye el identificador de Linear cuando corresponda y describe qué mejora obtiene el usuario o el sistema. No enumeres archivos, funciones o pasos de implementación en el título.

Evita:

TI2-37: Agregar AuthForm, query login, mutation login, schema y pruebas

Prefiere:

TI2-37: Permitir que los usuarios inicien sesión desde la Web

11.2. Descripción del Pull Request

Abre la descripción con una explicación sencilla del problema que originó el issue. Después explica la solución y añade los detalles necesarios para revisarla. No empieces con un inventario de archivos, funciones eliminadas o componentes modificados.

Evita:

Se agregó AuthForm, se creó la mutation login, se modificó schema.ts, se actualizaron los hooks y se añadieron pruebas para cada componente.

Prefiere:

El formulario de acceso no permitía iniciar sesión desde la Web. Ahora la aplicación valida las credenciales mediante el backend compartido y muestra los errores de autenticación sin duplicar la lógica del servidor.

El Pull Request debe indicar:

- qué problema resuelve;
- qué cambios introduce;

- cómo puede probarse;
- issue de Linear relacionado;
- cualquier consideración relevante para el reviewer.

Ejemplo:

TI2-37: Implementar login de usuarios

La descripción debe dar el contexto suficiente para revisar el cambio sin reconstruirlo desde el código.

12. Relación entre Issues y Pull Requests

Un issue y un Pull Request no tienen que mantener una relación 1:1.

Un issue representa una **unidad de trabajo**.

Un Pull Request representa una **unidad de integración de código**.

Un issue puede requerir varios Pull Requests.

Ejemplo:

TI2-42 Autenticación

```

|
+-- PR 1: modelo de usuario
|
+-- PR 2: endpoint login
|
+-- PR 3: refresh token

```

El issue termina cuando todo el trabajo necesario está integrado.

13. Stacked Pull Requests

Si no puedes dividir una tarea en cambios independientes, encadena Pull Requests o usa *stacked PRs*.

Ejemplo:

```

main
|
+-- PR A
    |
    +-- PR B
        |
        +-- PR C

```

Esta técnica es una excepción. No es el flujo predeterminado.

Como límite práctico:

usa como máximo 3 stacked PRs; el límite absoluto es 4.

Si una cadena supera cuatro Pull Requests dependientes, revisa la planificación y divide la tarea de nuevo.

14. Revisión de código

Otra persona debe revisar cada Pull Request antes de integrarlo.

Una revisión puede terminar en:

- aprobación;
- comentarios;
- solicitud de cambios.

La revisión debe comprobar:

- cumplimiento del issue;
- corrección;
- claridad del código;
- mantenibilidad;
- errores potenciales;
- pruebas;
- seguridad;
- respeto de la dirección de dependencias entre capas;
- ubicación correcta de las reglas de negocio;
- compatibilidad con otros componentes;
- contrato de API cuando corresponda.

15. Progresión de las revisiones de TI2

TI2 está en una etapa inicial de formación, así que la supervisión cambiará durante el semestre.

Etapla inicial

Durante la primera etapa, integrantes de TI4 revisarán los Pull Requests de TI2.

TI4 puede pedir cambios, explicar el problema y sugerir una solución. La persona autora hace los cambios en su propia rama.

Etapla intermedia

En la etapa intermedia, TI2 empezará a revisar código entre pares.

TI4 seguirá revisando directamente los cambios relacionados con:

- API;
- arquitectura;
- base de datos;
- seguridad;
- integración con móvil;
- cambios con impacto significativo.

Etapla avanzada

En la etapa avanzada, TI2 debería revisar e integrar sus cambios con autonomía.

TI4 se concentrará en:

- coordinación;
- QA;
- arquitectura;
- integración;
- resolución de dependencias.

16. Política de Merge

Usaremos **Squash and Merge**. Así, cada Pull Request deja un solo commit en **main**, aunque la branch tenga commits intermedios.

Después del merge:

- conserva la rama de desarrollo;
- actualiza el issue relacionado;
- incluye la documentación asociada.

Las ramas integradas forman parte del historial de trabajo y de la evidencia de participación de cada integrante.

17. Protección de main

TI4 mantendrá protegida la rama **main** en GitHub. Ustedes no tienen que configurar estas reglas cada vez que trabajen, pero sí deben conocerlas para saber por qué todos los cambios pasan por Pull Request.

GitHub bloqueará como mínimo:

- el push directo a **main**;
- el merge sin Pull Request;
- el merge sin al menos una aprobación;
- el merge cuando fallen las validaciones automáticas requeridas;
- el merge con conversaciones pendientes.

Las mismas reglas aplican a TI2 y TI4. Incluso los cambios de TI4 pasan por Pull Request para que el historial sea visible y ustedes puedan participar de la revisión.

18. Integración Continua

La integración continua es el filtro automático que les ayuda a detectar errores antes del merge. Cada Pull Request activa GitHub Actions y ustedes pueden revisar el resultado desde la pestaña de comprobaciones.

Las rutas modificadas determinan qué verificaciones se ejecutan. Como mínimo, esperamos que la CI:

- instale las dependencias desde el **package.json** y el único **bun.lock** de la raíz;
- ejecute el build de la aplicación o paquete afectado;
- aplique las herramientas de formato;
- ejecute el linter;
- ejecute las pruebas unitarias del Dominio y de los casos de uso;
- ejecute las pruebas de integración de Infraestructura y Presentación cuando corresponda.

Si su cambio afecta el backend, Web, Mobile o un paquete compartido, la CI revisa todos los componentes afectados.

Si la CI falla, revisen el log, corrijan el problema en su branch y vuelvan a publicar el cambio. No integren un Pull Request con la CI fallando salvo que TI4 acuerde una excepción justificada.

19. Integración entre TI2 y TI4

Ustedes, como TI2, mantienen la aplicación Web y los servicios compartidos. El detalle técnico de esas responsabilidades, la arquitectura y los adaptadores está en **tech.tex**; aquí

nos interesa cómo coordinar los cambios con TI4.

El monorepo permite que TI2 y TI4 coordinen una funcionalidad en el mismo Pull Request o separen los cambios en varios Pull Requests. Elijan la opción que facilite la revisión y el despliegue, y relacionen siempre los cambios en Linear.

Cuando un cambio de ustedes afecte una funcionalidad de TI4:

- relacionen el cambio con un issue de Linear;
- identifiquen al equipo afectado;
- comuniquen los cambios incompatibles antes de integrarlos;
- actualicen la documentación del servicio;
- indiquen si afecta Presentación, Aplicación, Dominio o Infraestructura;
- revisen la autenticación, los tipos y la estructura de datos cuando corresponda.

20. Cambios en el modelo de datos

Traten cualquier cambio del modelo de datos como un cambio que puede afectar a ambos equipos. La estructura técnica del modelo y su ubicación están explicadas en `tech.tex`; en este flujo nos importa que el cambio sea visible y coordinado.

Antes de integrarlo, registren el cambio en Linear, indiquen qué funcionalidades puede afectar y avisen a TI4 si requiere adaptar Mobile.

21. Dependencias entre equipos

Si su tarea depende del trabajo de TI4, o si TI4 queda esperando un cambio de ustedes, registren la dependencia en Linear. Esto permite que TI4 la siga y que ustedes sepan qué información o entrega falta.

Ejemplo:

TI4-25 Pantalla de recuperación de clave

Bloqueada por:

TI2-61 Endpoint de recuperación de clave

TI4 hará seguimiento de estas dependencias durante la coordinación del proyecto y les avisará si una de ellas bloquea el avance.

22. Definición de Done

Para TI4, un issue no termina porque “el código funciona en el computador del desarrollador”. Termina cuando otra persona puede revisar, integrar y entender el cambio dentro del proyecto.

Marquen una tarea como **Done** cuando:

- cumpla sus criterios de aceptación;
- el código esté integrado mediante Pull Request;
- las observaciones de la revisión estén resueltas;
- la CI haya finalizado correctamente;
- hayan realizado las pruebas necesarias;
- hayan actualizado la documentación cuando corresponda;
- las dependencias entre capas respeten la arquitectura definida;
- no queden dependencias conocidas sin registrar;
- el cambio esté disponible en `main`.

23. Flujo de trabajo completo

Este es el recorrido que esperamos que sigan para cada tarea:

1. TI4 prepara el issue y lo asigna al Team y al Cycle correspondientes.
2. Tomen la tarea desde Linear y revisen qué resultado se espera.
3. Identifiquen las capas afectadas y el punto de entrada de la funcionalidad.
4. Actualicen localmente `main` y creen desde Linear una branch asociada al issue dentro del monorepo.
5. Implementen el cambio y hagan commits en su branch.
6. Publiquen la branch en GitHub y abran el Pull Request.
7. Relacionen el Pull Request con el issue en Linear.
8. Revisen el resultado de la CI y respondan las observaciones del review.
9. Cuando CI y la revisión estén aprobadas, TI4 hará Squash and Merge.
10. Conserven la branch para mantener visible el historial de trabajo.
11. Actualicen el estado del trabajo en Linear.
12. Coordinen el despliegue cuando el cambio lo requiera.

El flujo queda así:

```
Linear
|
v
Issue
|
v
Branch pequeña
|
v
Desarrollo
|
v
Pull Request
|
+----> CI
|
+----> Code Review
|
v
Squash and Merge
|
v
main
|
v
Deploy
```

La branch se conserva para mantener visible el historial.

24. Referencia tecnológica

Este tutorial se ocupa del flujo de trabajo. El stack, la arquitectura, la estructura del monorepo, las variables de entorno y los despliegues están documentados en `tech.tex`.

Consulten esa guía cuando una tarea necesite una decisión técnica. Si la decisión afecta a ambos equipos, además de documentarla allí deben registrarla en Linear y coordinarla con TI4 mediante el flujo descrito en este documento.

25. Principios generales

Como TI4, esperamos que TI2 trabaje con estos principios:

1. Registren el trabajo relevante para que podamos seguirlo.
2. No hagan push directo a `main`.
3. Mantengan las ramas pequeñas y de corta duración.
4. Pidan una revisión antes de integrar el código.
5. Aprovechen la revisión de TI4 para ganar autonomía, no para delegar su trabajo.
6. Mantengan Linear actualizado para que refleje el estado real del trabajo.
7. Usen GitHub para revisar y mantener el código.
8. Mantengan todos los componentes principales del producto en el monorepo.
9. Separen Presentación, Aplicación, Dominio e Infraestructura.
10. Mantengan las reglas de negocio fuera de las aplicaciones cliente.
11. Mantengan explícitos y versionados la API compartida y los paquetes internos.
12. Avisennos pronto cuando encuentren un bloqueo o una decisión que no puedan resolver.

26. Resumen

Si tienen dudas durante el desarrollo, recuerden esta división:

Linear = planificación y seguimiento

GitHub = código, revisión, CI e integración

Monorepo = código compartido, trazable y coordinado

TI4 les entrega contexto, revisión y acompañamiento; TI2 implementa sus tareas y gana autonomía con cada ciclo. El objetivo es que ambos equipos puedan mantener el proyecto sin depender de una sola persona.